# Experiment No.3

**AIM**: To deploy multi-service applications using Docker Compose.

**THEORY:**

The objective of this experiment is to deploy and manage multi-service applications using Docker Compose, a powerful tool designed to define and run complex, multi-container Docker environments. Traditionally, launching multiple containers requires running individual, lengthy docker run commands with numerous flags for networking, ports, and environment variables. Docker Compose streamlines this entire process by utilizing a single YML configuration file—typically named docker-compose.yml—to orchestrate all application services, networks, and persistent storage volumes simultaneously.

Docker Compose is a tool that simplifies managing multi-container applications by using a single docker-compose.yml file to define services, networks, and storage. In this experiment, you create a project directory, configure a stack containing a Nginx web server and a MySQL database with persistent storage, and launch everything in the background using docker compose up -d. Once deployed, you can use built-in commands to easily monitor status, view logs, restart services, or clean up containers and volumes.

COMMANDS:

- **mkdir compose-demo** Creates a new directory named compose-demo on your file system to serve as a dedicated workspace. Organizing projects this way ensures Docker Compose can automatically locate your configuration file.
- **cd compose-demo** Changes the current terminal working directory into the newly created project folder. This ensures subsequent file creation and Docker commands execute within the correct project context.
- **dir** Lists the contents of the current directory to verify that files, such as the docker-compose.yml blueprint, have been created and saved correctly.
- **docker compose up** Builds, creates, and starts all services defined in the configuration file while streaming their combined output logs directly to your terminal. It keeps the terminal actively attached to the container processes.
- **docker compose up -d** Deploys and starts all services in detached mode, running the containers as background processes. This frees up your terminal window while the application stack runs seamlessly in the background.
- **docker compose ps** Displays a summary list of all currently running containers managed by the project's configuration file. It provides quick visibility into container health status and active port mappings.
- **docker compose logs** Aggregates and displays the log output from all services defined in the stack. This serves as an essential troubleshooting tool for diagnosing application errors or startup failures.
- **docker compose stop** Gracefully halts all running containers associated with the project without removing them from the system. This allows the application stack to be quickly brought back online later.

- **docker compose start** Restarts existing containers that have been previously stopped using the stop command. It brings the application stack back to life without needing to recreate the containers from scratch.
- **docker compose restart** Performs a quick stop-and-start cycle on all active containers within the stack. This is frequently used to refresh services or apply minor runtime adjustments.
- **docker compose down** Stops and completely removes all containers and internal networks created by the compose file. It cleans up the active environment while preserving persistent data volumes.
- **docker compose down -v** Performs a full teardown by stopping and removing containers, networks, and all associated persistent volumes. Use this with caution when you want to completely wipe database storage and reset the environment.
- **docker compose images** Lists an inventory of all the Docker images currently being utilized by the services defined in your configuration file.
- **docker compose config** Parses and validates the docker-compose.yml file for syntax errors and structural mistakes. It checks the configuration for correctness without starting any services.

**CODE:**

```
version: '3.8'
services:
  web:
    image: nginx
    container_name: webserver
    ports:
      - "8080:80"

  database:
    image: mysql:8.0
    container_name: mysqlserver

    environment:
      MYSQL_ROOT_PASSWORD: root123
      MYSQL_DATABASE: studentdb
      MYSQL_USER: student
```

**MYSQL_PASSWORD: student123**

**ports:**

**- "3306:3306"**

**volumes:**

**- mysql-data:/var/lib/mysql**

**volumes:**

**mysql-data:**

**EXECUTION:**

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Lab202\compose-demo>  dir


    Directory: C:\Users\Lab202\compose-demo


Mode                 LastWriteTime         Length
----                 -------------         ------
Mode                 LastWriteTime         Length

----                 -------------         ------

d-----         7/27/2026   3:32 PM                -a----         7/27/2026   3:32 PM            457


                                        PS C:\Users\Lab202\compose-demo> docker compose up -d
time="2026-07-27T15:36:40+05:30" level=warning msg="C:\\Users\\Lab202\\compose-demo\\docker-compose.yml: the attribute `version` is obsolete,
it will be ignored, please remove it to avoid potential confusion"[+] up 4/8                        ✓ Image mysql:8.0               Pulled
                                                15.1s ✓ Network compose-demo_default    Created
                 0.0s ✓ Volume compose-demo_mysql-data Created                                                  0.0s ✓ Container web
server           Started                                                 1.3s - Container mysqlserver          Starting
                                              1.3sError response from daemon: ports are not available: exposing port TCP 0.0.0.0:3306 -> 127.0
.0.1:0: listen tcp 0.0.0.0:3306: bind: Only one usage of each socket address (protocol/network address/port) is normally permitted.

What's next:
    Debug this Compose error with Gordon → docker ai "help me fix this compose erdocker compose ps C:\Users\Lab202\compose-demo>
time="2026-07-27T15:37:23+05:30" level=warning msg="C:\\Users\\Lab202\\compose-demo\\docker-compose.yml: the attribute `version` is obsolete,
it will be ignored, please remove it to avoid potential confusion"
NAME        IMAGE     COMMAND                  SERVICE   CREATED          STATUS          PORTS
webserver   nginx     "/docker-entrypoint.…"   web       27 seconds ago   Up 26 seconds   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
PS C:\Users\Lab202\compose-demo> docker compose logs
10:06:57 [notice] 1#1: start worker process 38
```

```
PS C:\Users\Lab202\compose-demo> docker compose logs
10:06:57 [notice] 1#1: start worker process 38
webserver  | 2026/07/27 10:06:57 [notice] 1#1: start worker process 39
webserver  | 2026/07/27 10:06:57 [notice] 1#1: start worker process 40
```

```
PS C:\Users\Lab202\compose-demo> docker compose stop
time="2026-07-27T15:43:59+05:30" level=warning msg="C:\\Users\\Lab202\\compose-demo\\docker-compose.yml: the attribute `version` is obsolete,
it will be ignored, please remove it to avoid potential confusion"
[+] stop 2/2
 ✓ Container mysqlserver Stopped                                                                              0.0s
 ✓ Container webserver   Stopped                                                                              0.3s
```

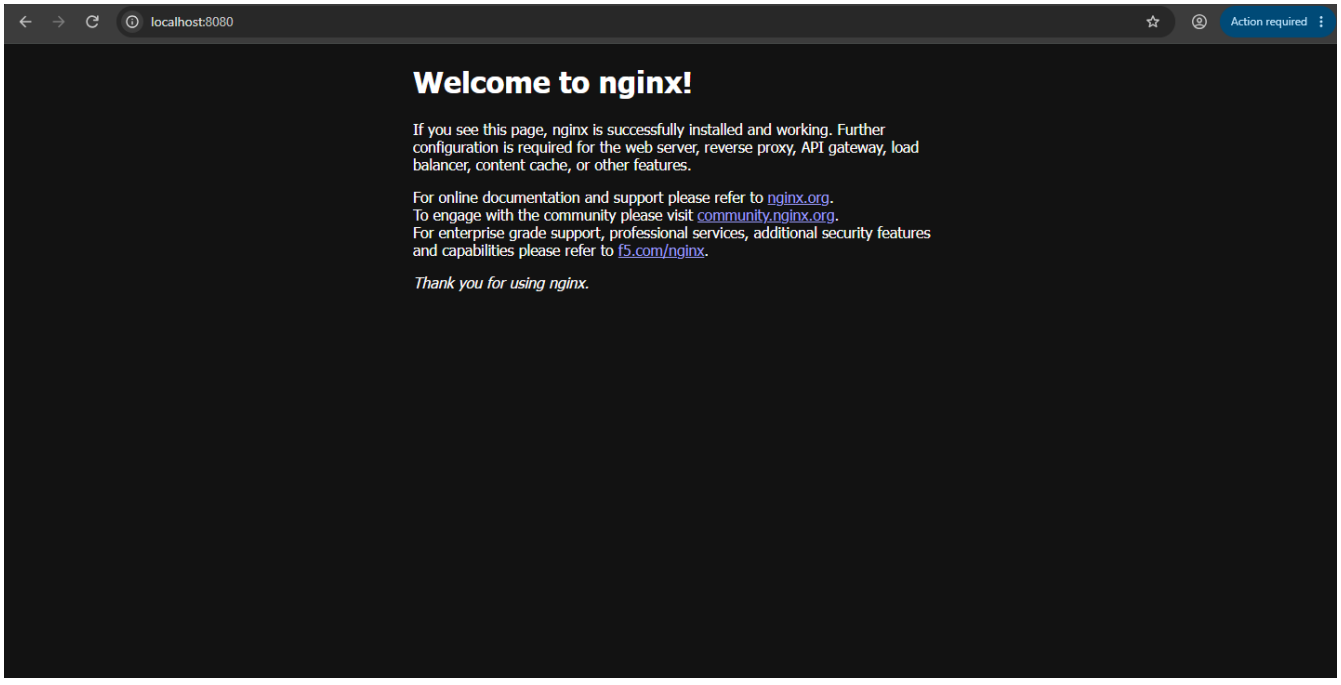
```
PS C:\Users\Lab202\compose-demo> docker compose start
time="2026-07-27T15:44:13+05:30" level=warning msg="C:\\Users\\Lab202\\compose-demo\\docker-compose.yml: the attribute `version` is obsolete,
it will be ignored, please remove it to avoid potential confusion"
[+] start 1/2
 ✓ Container webserver   Started                                                                                                    0.3s
 - Container mysqlserver Starting                                                                                                   0.3s
Error response from daemon: ports are not available: exposing port TCP 0.0.0.0:3306 -> 127.0.0.1:0: listen tcp 0.0.0.0:3306: bind: Only one us
age of each socket address (protocol/network address/port) is normally permitted.
```

```
PS C:\Users\Lab202\compose-demo> docker compose restart
time="2026-07-27T15:44:39+05:30" level=warning msg="C:\\Users\\Lab202\\compose-demo\\docker-compose.yml: the attribute `version` is obsolete
it will be ignored, please remove it to avoid potential confusion"
[+] restart 0/2
 - Container mysqlserver Restarting                                                                0.5s - Container webserver   Restarting
                        0.5sPS C:\Users\Lab202\compose-demo>
```

```
                        0.5sPS C:\Users\Lab202\compose-demo> docker compose down
time="2026-07-27T15:45:04+05:30" level=warning msg="C:\\Users\\Lab202\\compose-demo\\docker-compose.yml: the attribute `version` is obsolete,
it will be ignored, please remove it to avoid potential confusion"
[+] down 3/3
 ✓ Container webserver          Removed                                                0.4s ✓ Container mysqlserver        Removed
                        1.6s ✓ Network compose-demo_default Removed                                                    0.2sPS C:\Users\Lab202\compose
-demo>
```

```
-demo> docker compose down -v
time="2026-07-27T15:45:31+05:30" level=warning msg="C:\\Users\\Lab202\\compose-demo\\docker-compose.yml: the attribute `version` is obsolete,
it will be ignored, please remove it to avoid potential confusion"
[+] down 1/1
 ✓ Volume compose-demo_mysql-data Removed                                                  0.0sPS C:\Users\Lab202\compose-demo>
```

```
 ✓ Volume compose-demo_mysql-data Removed                                                  0.0sPS C:\Users\Lab202\compose-demo> docker compose images
time="2026-07-27T15:46:29+05:30" level=warning msg="C:\\Users\\Lab202\\compose-demo\\docker-compose.yml: the attribute `version` is obsolete,
it will be ignored, please remove it to avoid potential confusion"
CONTAINER           REPOSITORY          TAG                 PLATFORM            IMAGE ID            SIZE                CREATED
PS C:\Users\Lab202\compose-demo>
```



**CONCLUSION:**

This experiment successfully demonstrated the power and efficiency of Docker Compose in orchestrating multi-service application environments. By utilizing a single docker-compose.yml configuration file, complex multi-container architectures—such as pairing an Nginx web server with a persistent MySQL database—can be defined, deployed, and managed seamlessly with minimal commands. Through this hands-on process, the practical value of containerization, network configuration, and data persistence using volumes became clear, highlighting how Docker Compose eliminates manual overhead and ensures consistent, reproducible environments for modern software development.